C++

BY: JANHAVI DEO

Call by value and call by address

In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.

```
void change(int num) {
    cout<< “\nBefore adding value inside function num  = “ << num;
    num = num + 100;
    cout<< “\nAfter adding value inside function num = "<<num;
 }

int main() {
    int x=100;
     cout<<“\nBefore function call x  = “<<  x;
    change(x);//passing address in function
    cout<<“\nAfter function call x = “ <<  x;
    return 0;
}
```

Call by value and call by address

- In call by address, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- Any changes made using valueAt (*) operator on formal parameter will effect value of actual parameter in caller.

Call by address

```
void change(int *num) {
   cout<< “\nBefore adding value inside function num  = “ << *num;
   *num = *num + 100;
   cout<< “\nAfter adding value inside function num = "<< *num;
}

int main() {
   int x=100;
   cout<<“\nBefore function call x  = “<<  x;
   change(&x);//passing address in function
   cout<<“\nAfter function call x = “ <<  x;
return 0;
}
```

Pointers in C++

It is a variable, which holds address of another variable. The address should be from same process's address space.

Syntax: int *ptr;

Operators used with pointer: addressOf(&) and valueAt operators(*)

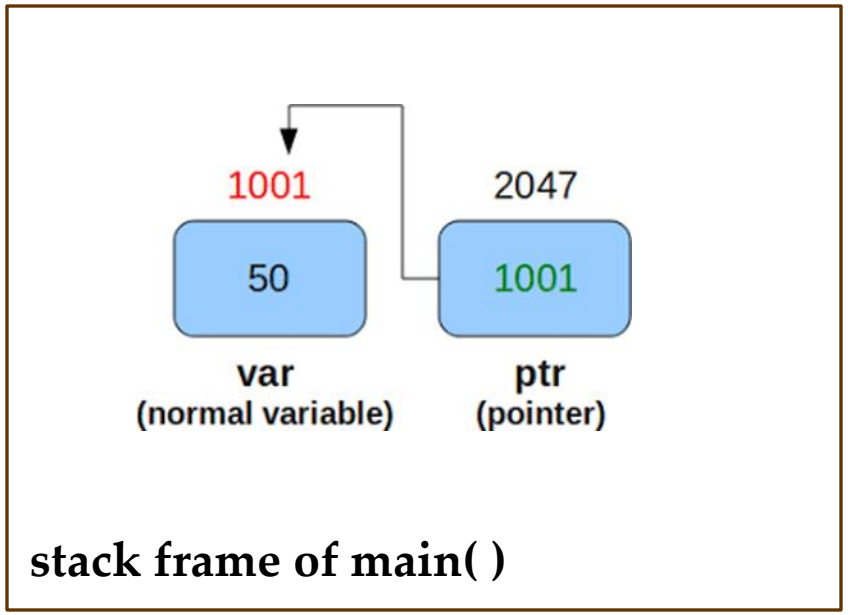
Usages:
1. To return more than one values from function. (call by address)
2. To store the address returned by new operator at run time.
3. Array is a constant pointer.
4. To access a variable which has been declared outside the function (declared in caller() etc)
5. To write fast and efficient programs.

Pointer comes with a lot of responsibility.

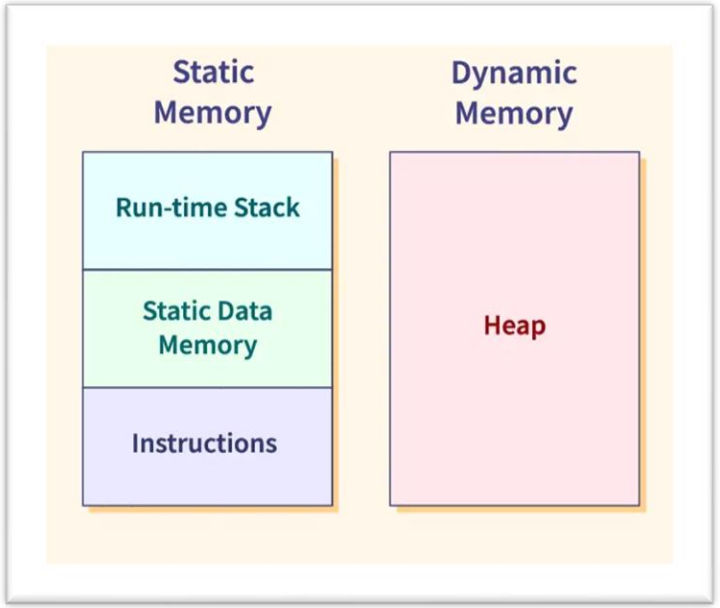
Sample code to understand **Pointers**

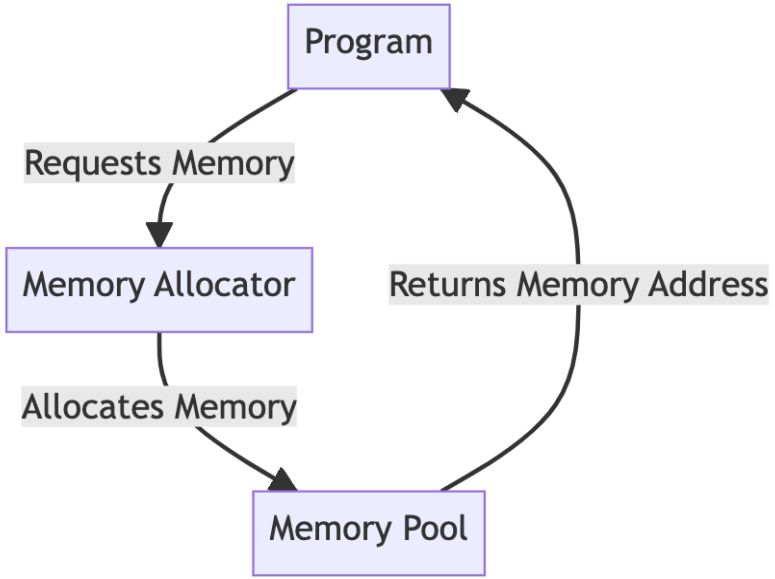
```
int main( )
{
    int var = 50;
    int *ptr = &var;

    cout <<  " \n var = " << var;
    cout <<  " \n Address of var = " << &var;
    cout <<  " \n ptr = "  << ptr;
    cout<<  "  \n Value at ptr = " << *ptr;
    cout << " \n Address of ptr = " <<  &ptr;
    return 0;
}
```



Dynamic Memory Allocation





new and delete operators in C++

- new operator is used to allocate memory on **heap** at run-time.
- It allocates memory, typecast it and returns address of allocated memory.

```
int *ptr = new int;

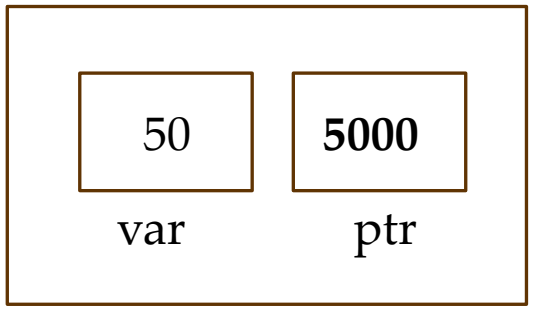
char *cptr = new char[5];
```

- delete operator is used to release allocated memory.
- pointer variable should be assigned to NULL once memory is released.

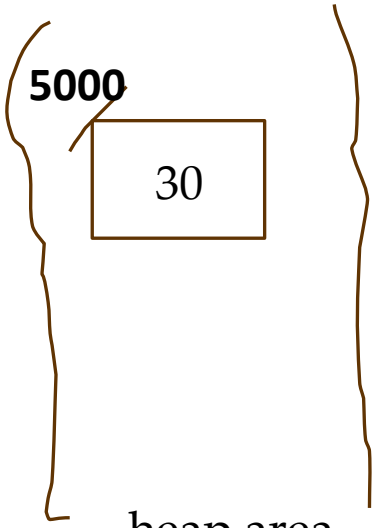
```
delete ptr;          delete [ ]cptr;
ptr = NULL;          cptr = NULL;
```

Sample code to understand Dynamic Memory Allocation

```
int main( )
{
      int var = 50;
      int *ptr;
      ptr = new int;
      *ptr = 30;
      cout<< " \n var = "<<var;
      cout<< " \n Address of var = "<<&var;
      cout<< " \n ptr = " <<ptr;
      cout<< " \n Value at ptr = "<<*ptr;
      cout<<" \n Address of ptr = "<<&ptr;
      return 0;
}
```



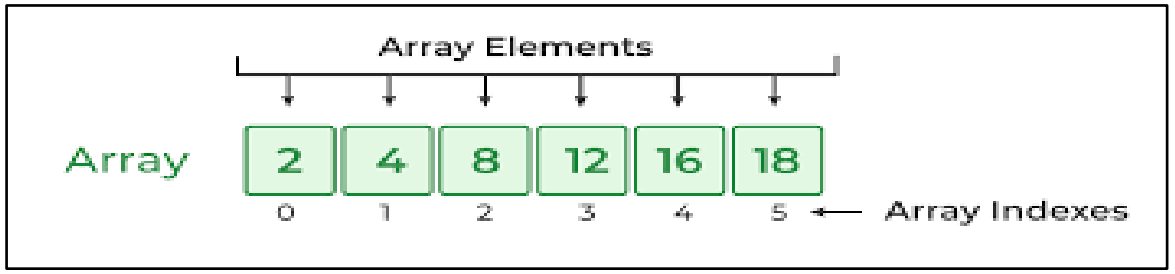
stack frame of main( )

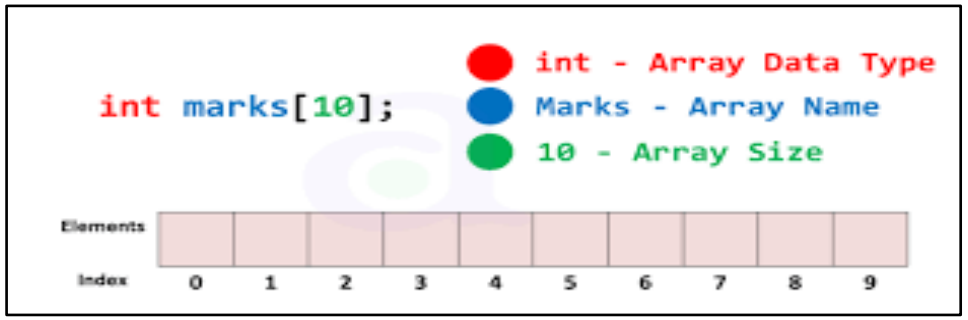


heap area

Array in C++: Arrays are used to store multiple values in a single variable.

1. It is a constant pointer.
2. collection of values of similar data type
3. contiguous memory allocation
4. fixed size
5. Array name itself is base address of array
6. Array is constant pointer
7. We can access array elements using index or subscript
8. Array index starts from 0 to size-1
9. It can be 1-d, 2-d, n-d
10. Random access is possible





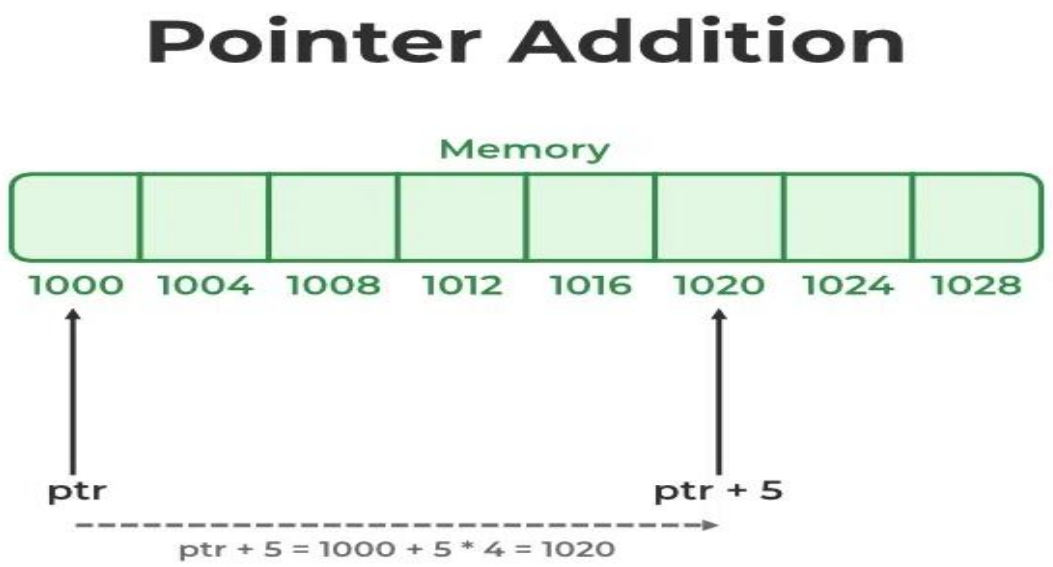
Syntax : datatype nameOfArray[size of array];

Pointer Arithmetic

Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers.

Pointer Arithmetic works on size of its datatype.

When added value 1 into an int pointer, it will get increased by 4 bytes.



Pointer and Array

- Array is a constant pointer.
- Array name itself is base address of array.
- The **memory address** of the **first element** is the same as the **name of the array**
- The operator valueAt (*) gives us value of first element. Rest all elements are accessed using pointer addition.
- Subscript operator is resolved by compiler.

  nameOfArray[n] ====> *(nameOfArray + n)

- Arrays are always pass by address and never pass by value

Sample code to understand **Array**

```
#include<iostream>
using namespace std;

int main()
{
        int arr[5];
        int i;
        cout<<"\n arr = "<<arr;

        for(i=0;i<5;i++)
        {
                cout<<"\nEnter number..";
                cin>>arr[i];
        }

        for(i=0;i<5;i++)
        {
           cout << "\n arr["<< i <<"] = “ << arr[i];
           cout << "\n *(arr +"<< i <<") = “ <<  *(arr+i);
           cout << "\n *("<< i <<" + arr ) = "<<  *(i+arr);
           cout << "\n "<< i << "[arr] = “ << i[arr];
        }
        return 0;
}
```

Assignments on Array:

- Find sum and average of array element.
- Find largest and smallest array element.
- Find largest and second largest array element.
- Shifting array elements to left and right.
- Insertion and deletion of array element from a given position
- Searching an element into an array
- Print array elements in reverse order.

Understand 2-d array and Complex pointer or pointer to array:

Array of arrays is 2-d array.

Syntax: int arr[3][3];
OR int (*ptr)[3];
OR int ptr[ ][3];

Demonstrate 2-d array – addition and multiplication of 2 matrices

Points to remember:

1. Array are never pass by value, they are always pass by address
2. C++ never checks for array index out of bound. It uses garbage value if program tries to go beyond index. The program may crash.

Dynamic memory allocation in C and C++

calloc ( )	malloc ( )	realloc ( )
It is used to dynamically allocate specified number of blocks of memory of the specified type	It is used to dynamically allocate a single large block of memory with the specified size.	It is used to reallocate memory, if previously allocated memory is insufficient
It returns a pointer of type void which can be cast into a pointer of any form.	It returns a pointer of type void which can be cast into a pointer of any form.	It returns a pointer of type void which can be cast into a pointer of any form.
It initialize memory at execution time to 0. It has two parameters or arguments as compare to malloc().	It doesn't Initialize memory at execution time so that it has initialized each block with the default garbage value initially.	Re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.
If space is insufficient, allocation fails and returns a NULL pointer.	If space is insufficient, allocation fails and returns a NULL pointer.	If space is insufficient, allocation fails and returns a NULL pointer.
ptr = (cast-type*)calloc(n, element-size);	***ptr = (int*) malloc(100 * sizeof(int));***	**ptr = (int *) realloc(ptr , 400*sizeof(int))**

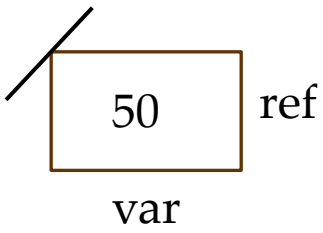
Reference variable:

- It is an alias declared for already declared variable in C++.
- No separate memory is allocated for reference variable.
- It must initialized at the time of declaration.
- Once a reference refers to a variable, it is stuck. It cannot be reassigned.
- It is used for passing parameters into function.

Syntax –

```
int var = 50;                 // integer variable
int &ref = var;               // reference to variable var

cout << "var = "<< var;    //print 50
cout << "ref = " << ref;    //print 50
```



Difference between pointer and reference variable

Comparison	Pointer	Reference

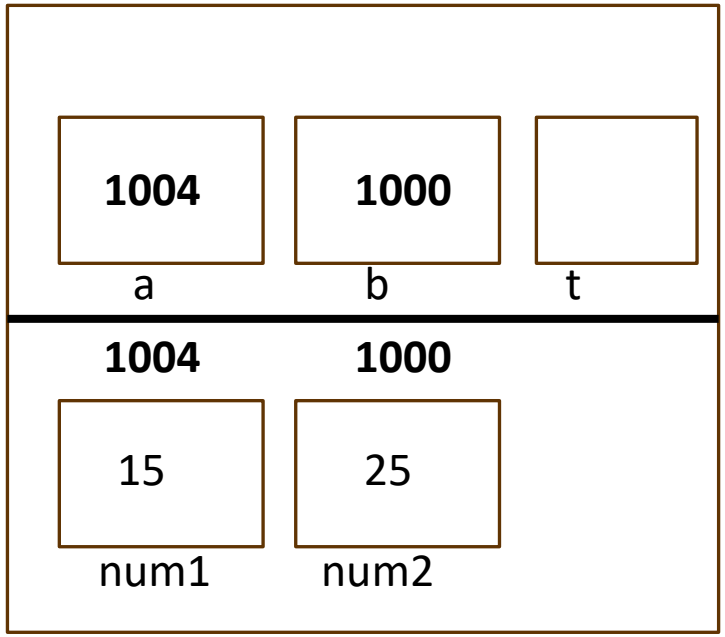
Initialization	Its not necessary to initialize the pointer at the time of declaration. Like
	
	int a = 10;
	int *P = &a; //It is not necessary
	Another way is :
	int a = 10;
	int *P;
	P = &a;
		Its necessary to initialize the Reference at the time of declaration. Like
		
		int num=10
		int &a = num;
		
		int &a; //Error here but not in case of Pointer.

Array	You can create the array of Pointer. int *p[5];	You can not create the Array of reference.
NULL value	You can assign NULL to the pointer like int *P = NULL; //Valid	You can not assign NULL to the reference like int &a = NULL; //Error
Pointer to pointer	You can use pointer to pointer.	You can not use reference to reference.

Sample code to understand call by address

```
void swap(int *, int *);   // function prototype
int main()
{
   int num1 = 15, num2 = 25;
   swap(&num1,&num2);  //function call
   cout << "\n After swap by address num1 = " << num1 << " and num2 = " << num2;
   return 0;
}

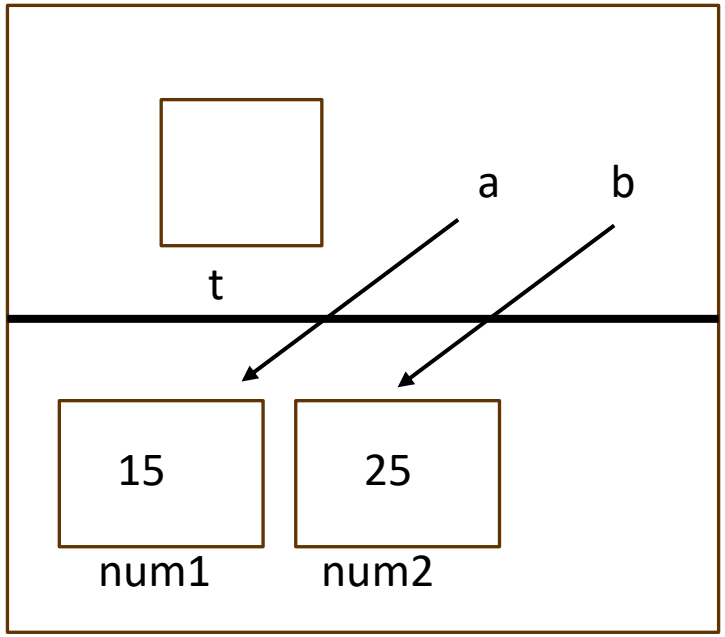
void swap(int *a, int *b)  // int *a = &num1    int *b = &num2
{
  int t;
  t = *a;
  *a = *b;
  *b = t;
}
```



Sample code to understand call by reference

```
void swap(int & ,int &);
int main()
{
   int num1 = 15, num2 = 25;
   swap(num1, num2);
   cout<<"\n After swap by reference num1 = "<<num1<<" and num2 = "<<num2;
   return 0;
}

void swap(int &a, int &b)  // int &a = num1    int &b = num2
{
        int t;
        t = a;
        a = b;
        b = t;
}
```



Function overloading

- We can write more than one function with the same name having different type and number of parameters.
- Used to improve readability of program, by function overloading we can design a family of functions that do essentially the same task but using different argument lists.
- Used for writing variety of constructors in a class.

Examples:

```
void print(const char * str);  // #1        call -       print("Infoway");
void print(double d);          // #2        call -       print(55.5);
void print(long i);            // #3        call -       print(5600L);
void print(int i, int j);      // #4        call -       print(55,67);
```

```
double cube(double x);          #1
double cube(double & x);        #2

        cout << cube(x);    // Ambiguous call
```

The x argument matches both the double x prototype and the double &x prototype

Note: Find out unsigned char and char are ambiguous or not ?

Function return type is not considered in function prototype

Function with default parameter value cannot be overloaded, if the number of parameters excluding default are same.

```
int sum(int a, int b = 20)
{
    return a + b;
}

/*int sum(int a)   //need to discard this definition
{
    return a+900;
}*/
```

```
int main()
{
    int num1= 10;
    cout<<"\n sum = "<<sum(num1);  //ambiguous
    return 0;
}
```

Sample code to understand **function overloading**

```
int sum(int a, int b, int c)
{
	return a+b+c;
}
int sum(int a, int b)
{
	return a+b;
}
float sum(float a , float b)
{
	return a + b;
}
float sum(float a, float b, float c)
{
	return a+b+c;
}
float sum(int n1, float n2)
{
	return (float)n1 + n2;
}
```

```
float sum(float n1, int n2)
{
	return n1 + (float)n2;
}

int main()
{
	cout<<"\n sum with 2 int...."<<sum(5,7);
	cout<<"\n sum with 3 int...."<<sum(6,7,8);
	cout<<"\n sum with 2 float..."<<sum(5.3f,2.5f);
	cout<<"\n sum with 1 int and 1 float...."<<sum(5,7.5f);
	cout<<"\n sum with 1 float and 1 int...."<<sum(5.5f,7);
	return 0;
}
```

Name Decoration or Name Mangling

To keep track of overloaded function, C++ compiler does name mangling.
The different functions with different signatures would result in a different name decorated by compiler.

D:\>objdump -d ExeFileName.exe > ObjectFileName.obj